

Leitura de Dados - Parte II

Análise de Dados

Aléssio Tony Cavalcanti de Almeida

Centro de Ciências Sociais Aplicadas
Universidade Federal da Paraíba
alessio@lema.ufpb.br



CIÊNCIA DE
DADOS PARA
NEGÓCIOS

João Pessoa, 2026

Plano de Aula

Objetivo: compreender diferentes estratégias de coleta e leitura de dados com Python, avançando da importação de arquivos locais para o acesso a dados por pacotes, APIs públicas, bancos de dados relacionais, bases NoSQL e arquivos volumosos.

Competências esperadas ao final da aula:

- reconhecer diferentes formas de disponibilização de dados em projetos aplicados;
- acessar dados por pacotes Python, APIs públicas e conexões com bancos de dados;
- transformar respostas de APIs, consultas SQL e documentos JSON em DataFrame;
- diagnosticar tipos, dimensões, valores ausentes e problemas básicos de leitura;
- adotar estratégias para bases volumosas, como seleção de colunas, definição de tipos, leitura em partes com `chunksize` e uso de formatos analíticos como Parquet;
- construir fluxos reprodutíveis de coleta, leitura, validação inicial e preparação de bases analíticas.

Estrutura da Aula

- 1 Introdução
- 2 Dados acessados por pacotes Python
- 3 APIs públicas
- 4 Bancos de dados relacionais
- 5 Bancos NoSQL: MongoDB
- 6 Dados volumosos
- 7 Atividade prática

Introdução

Retomada: de arquivos locais para fontes conectadas

O que já vimos

Na primeira parte da aula, trabalhamos com bases disponibilizadas como arquivos:

- CSV e TXT delimitado;
- TXT de largura fixa com dicionário;
- planilhas XLS/XLSX;
- arquivos DBC do DATASUS;
- inspeção inicial com `pandas`.

O que muda agora?

Em muitos projetos aplicados, os dados não chegam como um único arquivo baixado manualmente. Eles podem estar em APIs, pacotes Python, bancos relacionais, bancos NoSQL ou arquivos grandes demais para leitura integral.

Objetivo da segunda parte

Objetivo

Ampliar o repertório de coleta e leitura de dados em Python, saindo dos arquivos locais e avançando para fontes conectadas, bases institucionais e estratégias para dados volumosos.

Ao final desta parte da aula, espera-se que o estudante consiga

- identificar diferentes formas de disponibilização de dados;
- acessar dados por pacotes Python e APIs públicas;
- compreender a lógica de leitura a partir de bancos SQL e MongoDB;
- reconhecer limitações de memória e estratégias de leitura em partes;
- documentar a coleta para garantir reprodutibilidade.

A pergunta operacional

Como transformar uma fonte de dados externa em uma base analítica organizada?

Ideia central

Coletar dados não é apenas “baixar uma base”. É definir uma estratégia técnica compatível com a fonte, o volume, o formato, a frequência de atualização e a pergunta analítica.

Mapa das formas de acesso

Forma de acesso	Exemplos	Cuidados principais
Arquivo local	CSV, TXT, XLSX, DBC, Parquet	encoding, separador, tipos, dicionário
Pacote Python	Banco Mundial, Yahoo Finance, BCB	instalação, versão, documentação
API REST	BCB, IPEA, Portal da Transparência	URL, parâmetros, token, paginação
SQL	SQLite, PostgreSQL, MySQL	conexão, consulta, chaves, joins
NoSQL	MongoDB	documentos JSON, campos aninhados
Dados volumosos	RAIS, CNPJ/RFB, Censo Escolar, ENEM	memória, chunks, colunas, Parquet, Spark

Regra prática

A fonte define a estratégia de leitura. A pergunta analítica define quais dados realmente precisam ser carregados.

Critérios para escolher a estratégia de coleta

- 1 **Formato:** arquivo, API, tabela SQL, documento JSON ou base comprimida.
- 2 **Volume:** cabe ou não cabe na memória disponível?
- 3 **Frequência:** coleta única ou atualização recorrente?
- 4 **Granularidade:** indivíduo, vínculo, empresa, município, país, dia, mês?
- 5 **Governança:** há autenticação, token, restrição de uso ou documentação?
- 6 **Reprodutibilidade:** outra pessoa consegue executar a coleta?

Decisão técnica

O melhor método é aquele que resolve o problema com menor complexidade possível, mantendo rastreabilidade e validade.

Dados acessados por pacotes Python

Pacotes Python como conectores de dados

Ideia

Algumas bases de dados podem ser acessadas por pacotes Python que funcionam como conectores para APIs ou repositórios públicos.

Vantagens

- simplificam a chamada à fonte original;
- reduzem a necessidade de montar URLs manualmente;
- retornam dados próximos do formato tabular;
- aceleram prototipagem e ensino.

Cuidados

O pacote não elimina a necessidade de verificar metadados, códigos, unidades de medida, periodicidade e atualização da fonte.

Banco Mundial: que tipo de dado está disponível?

World Development Indicators

O Banco Mundial disponibiliza séries temporais internacionais sobre desenvolvimento econômico, social, demográfico, ambiental e institucional. A base *World Development Indicators* é uma das principais coleções, reunindo indicadores comparáveis entre países e ao longo do tempo.

Tipos de informação disponíveis

- **Economia:** PIB, PIB per capita, inflação, comércio, investimento.
- **População:** população total, crescimento populacional, urbanização.
- **Saúde:** expectativa de vida, mortalidade, vacinação.
- **Educação:** matrículas, conclusão, alfabetização.
- **Pobreza e desigualdade:** pobreza, renda, Gini.
- **Meio ambiente:** emissões, energia, saneamento, áreas protegidas.

Banco Mundial: cobertura, período e confiabilidade

Cobertura

A base WDI reúne indicadores para economias nacionais, agregados regionais e grupos de renda. A cobertura varia por indicador, país e período.

Período

Muitas séries possuem observações anuais desde a década de 1960, mas nem todos os indicadores têm cobertura completa para todos os países e anos.

Confiabilidade e limitações

Os dados são compilados a partir de fontes oficialmente reconhecidas, mas podem envolver estimativas, revisões metodológicas, ausência de dados e diferenças de qualidade estatística entre países. Portanto, a coleta deve sempre consultar os metadados do indicador.

Banco Mundial: como consultar a estrutura da base

Antes de baixar dados

O primeiro passo é explorar quais fontes, economias e indicadores estão disponíveis.

```
# pip install wbgapi pandas

import pandas as pd
import wbgapi as wb

# Listar bases ou fontes disponiveis
fontes = pd.DataFrame(wb.source.list())

fontes.head()
```

Leitura

Cada fonte representa uma coleção de dados. Para esta aula, o foco será a base de indicadores de desenvolvimento do Banco Mundial.

Banco Mundial: consultar países e agregados

Economias

O Banco Mundial usa códigos padronizados para países e agregados regionais ou econômicos.

```
economias = pd.DataFrame(wb.economy.list())  
  
economias.head()
```

```
economias[  
    economias["id"].isin(["BRA", "CHL", "MEX", "LCN", "WLD"])]
```

Atenção

Alguns códigos representam países; outros representam regiões ou agregados, como América Latina, mundo ou grupos por renda.

Banco Mundial: buscar indicadores por tema

Exemplo

Antes de escolher uma série, podemos buscar indicadores por palavras-chave.

```
indicadores_pib = pd.DataFrame(  
    wb.series.list(q="GDP per capita")  
)  
  
indicadores_pib[["id", "value"]].head(10)
```

```
indicadores_vida = pd.DataFrame(  
    wb.series.list(q="life expectancy")  
)  
  
indicadores_vida[["id", "value"]].head(10)
```


Banco Mundial: consultar metadados do indicador

Por que consultar metadados?

O código do indicador não basta. É preciso entender conceito, unidade, fonte original e limitações.

```
info_pib = wb.series.get("NY.GDP.PCAP.CD")
```

info_pib

```
info_vida = wb.series.get("SP.DYN.LE00.IN")
```

info_vida

Regra prática

Nunca usar um indicador internacional sem registrar seu código, nome, fonte, unidade de medida e período coletado.

Exemplo 1: Banco Mundial com wbgapi

Problema analítico

Comparar PIB per capita e expectativa de vida entre Brasil, Chile e México.

```
# pip install wbgapi pandas
import pandas as pd
import wbgapi as wb
indicadores = {"NY.GDP.PCAP.CD": "pib_per_capita",
               "SP.DYN.LE00.IN": "expectativa_vida"}

dados_wb = wb.data.DataFrame(
    list(indicadores.keys()),
    economy=["BRA", "CHL", "MEX"],
    time=range(2000, 2024),
    labels=True)

dados_wb.head()
```

Banco Mundial: organizando o retorno

```
df_wb = (dados_wb
         .reset_index()
         .rename(columns=indicadores))

df_wb.columns = (df_wb.columns
                 .str.lower()
                 .str.replace(" ", "_")
                 .str.replace(".", "_", regex=False))

df_wb.head()
```

Discussão didática

Mesmo usando um pacote pronto, ainda é necessário validar nomes, tipos, períodos, países, unidades de medida e valores ausentes.

Banco Mundial: inspeção inicial

```
df_wb.info()  
df_wb.isna().sum()  
  
df_wb.describe(include="all")
```

Perguntas para os alunos

- Há países ou anos sem dados?
- O indicador está em nível, taxa, percentual ou moeda corrente?
- Os países são comparáveis diretamente?
- Há necessidade de deflacionar ou transformar variáveis?

- **Preços históricos:** abertura, máxima, mínima, fechamento, volume e preço ajustado.
- **Ações:** empresas brasileiras e internacionais, usando códigos como PETR4.SA, VALE3.SA e AAPL.
- **Índices:** Ibovespa, S&P 500, Nasdaq, entre outros.
- **ETFs e fundos:** ativos negociados em bolsa.
- **Criptomoedas:** pares como BTC-USD e ETH-USD.
- **Informações cadastrais:** setor, mercado, moeda, nome do ativo e bolsa de negociação.

Os dados do Yahoo Finance são úteis para ensino, prototipagem e análise exploratória, mas exigem validação para aplicações profissionais. É necessário verificar ticker, liquidez, moeda, ajuste de preços, feriados, dados ausentes e possíveis mudanças na disponibilidade da fonte.

```
# pip install yfinance pandas
import pandas as pd
import yfinance as yf

ticker = yf.Ticker("PETR4.SA")

info = ticker.info

campos = ["symbol", "shortName", "currency",
"exchange", "quoteType", "sector", "industry"]

metadados = {campo: info.get(campo) for campo in campos}
pd.DataFrame([metadados])
```

Exemplo 2: dados financeiros com yfinance

Problema analítico

Obter preços históricos de ativos financeiros para calcular retorno diário.

```
# pip install yfinance pandas

import pandas as pd
import yfinance as yf

ativos = ["PETR4.SA", "VALE3.SA", "^BVSP"]

precos = yf.download(
    tickers=ativos,
    start="2020-01-01",
    end="2024-12-31",
    auto_adjust=True,
    progress=False
)
```


Yahoo Finance: retornos diários

```
fechamento = precos["Close"]

retornos = fechamento.pct_change().dropna()

resumo = retornos.describe().T

resumo
```

Cuidados

Dados financeiros exigem atenção a feriados, ativos sem negociação, preço ajustado, mudanças de ticker e eventos corporativos.

Yahoo Finance: tabela analítica simples

```
retornos_long = (
    retornos
    .reset_index()
    .melt(
        id_vars="Date",
        var_name="ativo",
        value_name="retorno_diario"
    )
)

retornos_long.head()
```

Boa prática

Para análise, modelagem e gráficos, muitas vezes é melhor trabalhar no formato longo: uma linha por data e ativo.

APIs públicas

O que é uma API?

Definição prática

Uma API permite que um programa acesse dados ou serviços de outro sistema por meio de requisições estruturadas.

Fluxo mais comum

- 1 montar uma URL com parâmetros e enviar uma requisição HTTP;
- 2 receber uma resposta em JSON;
- 3 transformar o JSON em DataFrame;
- 4 validar tipos, códigos, datas e valores; e salvar ou documentar o resultado.

Atenção

APIs podem exigir autenticação, impor limite de chamadas, alterar campos, exigir paginação ou retornar mensagens de erro.

Exemplo 3: API do Banco Central – SGS

Problema analítico

Baixar a série da taxa Selic diária diretamente do Banco Central.

```
import pandas as pd
import requests

codigo_serie = 11

url = (
    "https://api.bcb.gov.br/dados/serie/bcdata.sgs."
    f"{codigo_serie}/dados?formato=json"
)

resposta = requests.get(url, timeout=30)
resposta.raise_for_status()

selic = pd.DataFrame(resposta.json())
```

Banco Central: filtrando período

```
data_inicial = "01/01/2020"
data_final = "31/12/2024"
url = ("https://api.bcb.gov.br/dados/serie/bcdata.sgs."
      f"{codigo_serie}/dados"
      f"?formato=json&dataInicial={data_inicial}&dataFinal={data_final}")

resposta = requests.get(url, timeout=30)
resposta.raise_for_status()
selic_perodo = pd.DataFrame(resposta.json())

selic_perodo["data"] = pd.to_datetime(
    selic_perodo["data"], dayfirst=True)

selic_perodo["valor"] = pd.to_numeric(
    selic_perodo["valor"], errors="coerce")

selic_perodo.head()
```

Banco Central: alternativa com python-bcb

Alternativa

Em vez de montar a URL manualmente, é possível usar um pacote que encapsula a API.

```
# pip install python-bcb

from bcb import sgs

series_bcb = sgs.get(
    {
        "selic": 11,
        "ipca": 433
    },
    start="2020-01-01",
    end="2024-12-31"
)

series_bcb.head()
```

Exemplo 4: Ipeadata

Problema analítico

Consultar séries econômicas do Ipeadata diretamente no Python.

```
# pip install ipeadatapy

import pandas as pd
import ipeadatapy as ipea

metadados = ipea.metadata()

metadados.head()
```

Primeiro passo

Antes de baixar uma série, é preciso descobrir o código correto e conferir metadados.

Ipeadata: buscando séries por termo

```
pib_series = metadados[  
    metadados["NAME"]  
    .str.contains("PIB", case=False, na=False)  
]  
  
pib_series[["CODE", "NAME"]].head(10)
```

Cuidados

Séries podem diferir por fonte, periodicidade, unidade de medida, metodologia e atualização.

Ipeadata: coletando uma série

```
codigo = "BM12_PIB12"  
  
serie_ipea = ipea.timeseries(codigo)  
  
serie_ipea.head()
```

Validação mínima

Verificar data, valor, unidade, periodicidade, fonte primária e existência de quebras metodológicas.

Exemplo 5: Portal da Transparência

Problema analítico

Consultar dados públicos do governo federal via API.

Atenção

A API do Portal da Transparência exige chave de acesso. Nunca publicar token em notebook, slide ou repositório.

```
import os
import pandas as pd
import requests

token = os.getenv("PORTAL_TRANSPARENCIA_TOKEN")

url = "https://api.portaldatransparencia.gov.br/api-de-dados/orgaos-
      siafi"

headers = {
```

APIs: checklist de coleta reprodutível

- 1 Registrar a URL base e o endpoint utilizado.
- 2 Documentar parâmetros: período, localidade, código da série, página.
- 3 Tratar erro de conexão com `raise_for_status()`.
- 4 Controlar paginação quando houver muitas respostas.
- 5 Salvar uma cópia bruta da resposta, quando necessário.
- 6 Converter tipos: datas, números, códigos e categorias.
- 7 Criar log de coleta com data e hora da execução.

Regra prática

Uma coleta via API deve ser reproduzível por outra pessoa, em outro computador, sem depender de cliques manuais.

Bancos de dados relacionais

Quando usar SQL?

Situação típica

Bases institucionais, em geral, estão em várias tabelas relacionadas por chaves.

Exemplo

- estudantes;
- cursos;
- matrículas;
- componentes curriculares;
- notas;
- docentes.

Papel do SQL

Selecionar, filtrar, agregar e relacionar tabelas antes de carregar os dados no pandas.

SQL antes do pandas

Tarefa	No banco SQL	No pandas
Filtrar milhões de linhas	recomendado	possível, mas pesado
Fazer joins grandes	recomendado	possível, mas usa RAM
Criar agregações iniciais	recomendado	recomendado após redução
Análise exploratória	limitado	recomendado
Gráficos e modelos	não é o foco	recomendado

Mensagem

Use o banco para reduzir e organizar. Use o pandas para analisar, transformar e comunicar.

Exemplo 6: SQLite em memória

Base de demonstração

SQLite é útil para aula porque não exige servidor. O exemplo abaixo cria tabelas simples e executa uma consulta SQL.

```
import sqlite3
import pandas as pd

con = sqlite3.connect(":memory:")

clientes = pd.DataFrame({"id_cliente": [1, 2, 3, 4],
                        "uf": ["PB", "PB", "PE", "RN"]})

vendas = pd.DataFrame({"id_venda": [101, 102, 103, 104, 105],
                      "id_cliente": [1, 1, 2, 3, 4],
                      "valor": [120.0, 80.0, 250.0, 90.0, 140.0]})

clientes.to_sql("clientes", con, index=False, if_exists="replace")
vendas.to_sql("vendas", con, index=False, if_exists="replace")
```


SQLite: lendo uma consulta com pandas

```
consulta = """
SELECT
    c.uf, COUNT(v.id_venda) AS n_vendas,
    SUM(v.valor) AS valor_total,
    AVG(v.valor) AS valor_medio
FROM vendas v
JOIN clientes c
    ON v.id_cliente = c.id_cliente
GROUP BY c.uf
ORDER BY valor_total DESC
"""

df_sql = pd.read_sql_query(consulta, con)
df_sql
```

Lição

O pandas não precisa ler todas as tabelas. Ele pode ler apenas o resultado analítico de uma consulta SQL.

Exemplo 7: conexão com PostgreSQL

Situação real

Em projetos reais, os dados podem estar em PostgreSQL, MySQL, SQL Server etc.

```
# pip install sqlalchemy psycopg2-binary
import pandas as pd
from sqlalchemy import create_engine, text
engine = create_engine(
    "postgresql+psycopg2://usuario:senha@localhost:5432/meu_banco")
consulta = text("""
SELECT ano, uf, COUNT(*) AS n_registros
FROM minha_tabela
WHERE ano BETWEEN 2020 AND 2024
GROUP BY ano, uf
ORDER BY ano, uf
""")
df = pd.read_sql(consulta, engine)
df.head()
```

Boas práticas em SQL + pandas

- ❶ Evitar `SELECT *` em bases grandes.
- ❷ Selecionar apenas as colunas necessárias.
- ❸ Filtrar período e localidade no banco.
- ❹ Fazer agregações iniciais no SQL.
- ❺ Preservar chaves e códigos como texto quando necessário.
- ❻ Documentar a consulta usada para gerar a base.

Erro comum

Carregar a tabela inteira no pandas antes de saber quais variáveis serão usadas.

Bancos NoSQL: MongoDB

SQL versus NoSQL

Aspecto	SQL relacional	MongoDB / NoSQL
Estrutura	tabelas, linhas e colunas	documentos JSON/BSON
Esquema	mais rígido	mais flexível
Relações	joins e chaves	documentos aninhados
Uso típico	cadastros, transações, registros administrativos	logs, eventos, textos, dados semiestruturados
Leitura no pandas	read_sql	find() + DataFrame

Mensagem didática

NoSQL não substitui SQL. Ele resolve problemas diferentes, especialmente quando os dados têm estrutura flexível ou semiestruturada.

Exemplo 8: MongoDB com PyMongo

Demonstração local

O exemplo abaixo pressupõe um MongoDB local em execução na porta 27017.

```
import pandas as pd
from pymongo import MongoClient
client = MongoClient("mongodb://localhost:27017/")
db = client["aula_dados"]
colecacao = db["eventos"]
colecacao.delete_many({})
colecacao.insert_many([{"id": 1, "tipo": "acesso", "uf": "PB",
"valor": 10, "origem": {"sistema": "web"}},
{"id": 2, "tipo": "compra", "uf": "PB", "valor": 120,
"origem": {"sistema": "app"} },
{"id": 3, "tipo": "acesso", "uf": "PE", "valor": 20,
"origem": {"sistema": "web"}}])
```

MongoDB: consultando documentos

```
cursor = colecao.find(  
    {"uf": "PB"},  
    {"_id": 0, "id": 1, "tipo": 1,  
     "uf": 1, "valor": 1, "origem.sistema": 1})  
  
registros = list(cursor)  
  
df_mongo = pd.DataFrame(registros)  
  
df_mongo.head()
```

MongoDB: normalizando campos aninhados

```
cursor = colecao.find({}, {"_id": 0})

registros = list(cursor)

df_mongo = pd.json_normalize(registros)

df_mongo.head()
```

Cuidados

Campos aninhados e listas exigem decisões de modelagem: manter como lista, expandir em linhas ou criar tabelas auxiliares.

Quando MongoDB faz sentido?

Cenários típicos

- registros de eventos de sistemas;
- logs de acesso;
- documentos com campos variáveis;
- respostas de formulários com estrutura flexível;
- dados textuais ou semiestruturados;
- protótipos de aplicações com evolução rápida do esquema.

Para análise

Na maior parte dos casos, será preciso transformar documentos em uma base tabular antes de calcular indicadores, estimar modelos ou gerar visualizações.

Dados volumosos

- o arquivo demora muito para abrir e/ou o notebook trava ou fecha;
- a memória RAM fica próxima do limite;
- há muitas colunas que não serão usadas;
- a análise exige empilhar vários anos ou vários arquivos.

O erro mais comum é tentar carregar a base inteira antes de definir claramente quais variáveis e agregações são necessárias.

Exemplo 9: prototipagem com microdados da RAIS

Problema

Antes de processar toda a RAIS, podemos ler apenas algumas colunas e uma amostra inicial.

```
import pandas as pd
arquivo_rais = "RAIS_VINC_PUB_NORDESTE.txt"
colunas = ["Ano", "Município", "CNAE 2.0 Classe", "CBO Ocupacao 2002",
"Vl Remun Media Nom", "Qtd Hora Contr", "Vinculo Ativo 31/12"]

tipos = {"Ano": "int16", "Município": "string",
"CNAE 2.0 Classe": "string", "CBO Ocupacao 2002": "string",
"Vl Remun Media Nom": "string", "Qtd Hora Contr": "string",
"Vinculo Ativo 31/12": "category"}

rais_amostra = pd.read_csv(arquivo_rais, sep=";",
encoding="latin1", usecols=colunas, dtype=tipos, nrows=100_000 )

rais_amostra.head()
```

RAIS: ajuste de tipos numéricos

Problema frequente

Em bases públicas brasileiras, números podem vir como texto, com vírgula decimal ou caracteres especiais.

```
rais_amostra["Vl Remun Media Nom"] = (  
    rais_amostra["Vl Remun Media Nom"].str.replace(",", ".", regex=  
        False))  
  
rais_amostra["Vl Remun Media Nom"] = pd.to_numeric(  
    rais_amostra["Vl Remun Media Nom"], errors="coerce")  
  
rais_amostra["Qtd Hora Contr"] = pd.to_numeric(  
    rais_amostra["Qtd Hora Contr"], errors="coerce")  
  
rais_amostra.info(memory_usage="deep")
```

```
import pandas as pd
arquivo_rais = "RAIS_VINC_PUB_NORDESTE.txt"
colunas = ["Município", "Vl Remun Media Nom", "Vinculo Ativo 31/12"]

resultados = []

for parte in pd.read_csv(
    arquivo_rais, sep=";", encoding="latin1",
    usecols=colunas, dtype="string", chunksize=500_000 ):
    parte["Vl Remun Media Nom"] = (parte["Vl Remun Media Nom"]
        .str.replace(",", ".", regex=False))
    parte["remuneracao"] = pd.to_numeric(parte["Vl Remun Media Nom"],
        errors="coerce")

# Continua...
```

RAIS: processamento em partes

Problema

Calcular salário médio e vínculos ativos por município sem carregar toda a base em memória.

```
# Continuacao dentro do looping...
parte = parte[parte["Vinculo Ativo 31/12"] == "1"]

resumo_parte = (parte
    .dropna(subset=["remuneracao"])
    .groupby("Municipio")
    .agg(
        soma_remuneracao=("remuneracao", "sum"),
        n_vinculos=("remuneracao", "count")).reset_index())

resultados.append(resumo_parte)

acumulado = pd.concat(resultados, ignore_index=True)
```


RAIS: consolidando o resultado

```
rais_municipio = (  
    acumulado  
    .groupby("Município")  
    .agg(  
        soma_remuneracao=("soma_remuneracao", "sum"),  
        n_vinculos=("n_vinculos", "sum")  
    )  
    .assign(  
        remuneracao_media=lambda x: (  
            x["soma_remuneracao"] / x["n_vinculos"]  
        )).reset_index()  
  
rais_municipio.head()
```

Lógica

Cada parte gera uma agregação parcial. Depois, as agregações parciais são somadas para produzir o resultado final.

Use CSV para intercâmbio e Parquet para processamento analítico recorrente.

Exemplo 10: base cadastral de CNPJs da Receita Federal

Problema

A base de CNPJs da Receita Federal é disponibilizada em múltiplos arquivos. Em vez de carregar tudo, podemos processar arquivo por arquivo.

```
import pandas as pd
from pathlib import Path
pasta_cnpj = Path("dados_cnpj")
arquivos_estab = sorted(pasta_cnpj.glob("*ESTABELE*.csv"))

colunas = ["cnpj_basico", "cnpj_ordem",
            "cnpj_dv", "identificador_matriz_filial",
            "situacao_cadastral", "data_inicio_atividade", "
                                     cnae_fiscal_principal",
            "uf", "municipio"]

len(arquivos_estab)
```

CNPJ/RFB: agregando por UF e situação

```
resultados = []

for arquivo in arquivos_estab:
    leitor = pd.read_csv(
        arquivo,
        sep=";",
        encoding="latin1",
        header=None,
        dtype="string",
        chunksize=500_000
    )

    for chunk in leitor:
        chunk = chunk.iloc[:, :len(colunas)]
        chunk.columns = colunas

    resumo = (
        chunk
        .groupby(["UF", "situacao"])
        .agg({"valor": "sum", "qtd": "count"})
        .reset_index()
```

CNPJ/RFB: criando identificador completo

Boa prática

Em bases cadastrais, códigos devem ser tratados como texto, não como número. Isso evita perda de zeros à esquerda.

```
chunk["cnpj"] = (  
    chunk["cnpj_basico"].str.zfill(8)  
    + chunk["cnpj_ordem"].str.zfill(4)  
    + chunk["cnpj_dv"].str.zfill(2)  
)  
  
chunk[["cnpj", "uf", "situacao_cadastral"]].head()
```

Atenção

Nunca converter CNPJ, CPF, CEP, código de município ou código de escola para inteiro quando o objetivo for identificação.

CNPJ/RFB: agregando por UF e CNAE

Problema

Construir uma tabela analítica com quantidade de estabelecimentos ativos por UF e CNAE principal.

```
resultados = []

for arquivo in arquivos_estab:
    leitor = pd.read_csv(
        arquivo,
        sep=";",
        encoding="latin1",
        header=None,
        dtype="string",
        chunksize=500_000
    )

    for chunk in leitor:
        chunk = chunk.iloc[:, :len(colunas)]
```

RAIS e CNPJ: decisões práticas

Decisão	RAIS	CNPJ/RFB
Unidade de análise	vínculo empregatício	estabelecimento ou empresa
Chaves principais	município, CBO, CNAE, ano	CNPJ básico, ordem, DV, CNAE, município
Risco comum	carregar colunas demais	perder zeros à esquerda
Estratégia inicial	selecionar variáveis e anos	processar arquivos em lote
Saída recomendada	agregados por município/CBO/CNAE	agregados por UF/município/CNAE
Formato intermediário	Parquet	Parquet

Mensagem didática

O mais importante não é “abrir a base”, mas definir a unidade de análise, selecionar variáveis relevantes e criar uma tabela analítica consistente.

Quando considerar Spark?

Use Spark quando

- o dado não cabe em memória e há muitos arquivos grandes;
- as agregações são recorrentes e o processamento precisa ser distribuído;

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
spark = SparkSession.builder.appName("aula-dados").getOrCreate()

df_spark = (spark.read.option("header", True).option("sep", ";")
            .csv("RAIS_VINC_PUB_NORDESTE.txt"))

resumo = (df_spark.groupBy("Município")
          .agg(F.count("*").alias("n_registros")))

resumo.show()
```


Pandas, banco de dados ou Spark?

Situação	Ferramenta recomendada	Justificativa
Arquivo pequeno ou médio	pandas	simples, rápido e suficiente
Arquivo grande, mas agregável em partes	pandas com chunksize	reduz uso de memória
Vários arquivos anuais	pandas + Parquet	melhora reprocessamento
Muitas tabelas relacionadas	SQL + pandas	banco filtra e agrega melhor
Dados semiestruturados	MongoDB + pandas	flexibilidade documental
Dados massivos ou distribuídos	Spark	processamento paralelo
Pipeline institucional recorrente	banco + orquestração	governança e reprodutibilidade

Regra prática

Comece simples. Aumente a complexidade apenas quando houver gargalo real de volume, velocidade, variedade ou governança.

Atividade prática

Atividade prática: mapa de estratégias de coleta de dados volumosos

Objetivo

Escolher a estratégia de coleta e leitura adequada para diferentes problemas de dados.

- 1 Escolha uma fonte de dados que tenha um grande volume de dados (pelo menos 1GB), como RAIS, CNPJ/RFB, Microdados do INEP (como Enem), etc.
- 2 Definir uma pergunta analítica simples.
- 3 Fazer um processo de leitura eficiente dos dados em Python/Pandas usando a estratégias de chunks.
- 4 Transformar o resultado em um DataFrame organizado.
- 5 Apresentar uma tabela-resumo com 3 a 5 linhas de evidência.

Entrega da atividade

Produto esperado

Um notebook curto contendo:

- pergunta analítica;
- fonte de dados;
- estratégia de acesso;
- código de coleta;
- inspeção inicial com `head()`, `info()` e `describe()`;
- tabela final organizada;
- comentário sobre limitações da fonte.

Critério de avaliação

O foco não é fazer uma análise sofisticada. O foco é demonstrar domínio da coleta, leitura, validação inicial e documentação da fonte.

Roteiro mínimo do notebook

- 1 **Contexto:** qual problema ou pergunta será respondida?
- 2 **Fonte:** onde os dados estão disponíveis?
- 3 **Coleta:** qual método será usado?
- 4 **Leitura:** como transformar a fonte em DataFrame?
- 5 **Validação:** quais problemas iniciais aparecem?
- 6 **Tabela analítica:** qual base final será usada?
- 7 **Síntese:** o que já é possível afirmar?

Mensagem

Um bom notebook de coleta deve permitir que outra pessoa reproduza o mesmo resultado sem adivinhações.

Síntese da aula

- 1 Nem todo dado chega como CSV.
- 2 Pacotes Python simplificam o acesso, mas não eliminam validação.
- 3 APIs exigem atenção a parâmetros, autenticação, paginação e estabilidade.
- 4 Bancos SQL são fundamentais quando há múltiplas tabelas relacionadas.
- 5 Bancos NoSQL são úteis para dados semiestruturados.
- 6 Dados volumosos exigem estratégia: selecionar colunas, definir tipos, usar chunks e considerar Parquet ou Spark.

Mensagem final

Coletar dados não é uma etapa mecânica: é uma decisão metodológica que afeta validade, reprodutibilidade e escalabilidade da análise.

Encerramento

Antes de analisar, precisamos garantir que sabemos de onde o dado veio, como foi coletado e como foi transformado.

Próximo passo

Depois da coleta e leitura, o próximo desafio é transformar dados brutos em uma base analítica validada, documentada e pronta para exploração.

Referências I

 AMARAL, F. *Introdução à Ciência de Dados: mineração de dados e big data*. Rio de Janeiro: Alta Books, 2016.

 CARVALHO, A.; MENEZES, A.; BONIDIA, R. *Ciência de Dados: Fundamentos e Aplicações*. Rio de Janeiro: LTC, 2024. 1–354 p.

 FONTELLES, M. J. et al. Metodologia da pesquisa científica: diretrizes para a elaboração de um protocolo de pesquisa. *Revista Paraense de Medicina*, v. 23, n. 3, p. 1–8, 2009.

 IDRIS, I. *Python Data Analysis*. [S.l.]: Packt Publishing, 2014. ISBN 9781783553358.

 JAKE, V. *Python Data Science Handbook*. [S.l.: s.n.], 2019. v. 53. 1689–1699 p. ISSN 1098-6596. ISBN 9788578110796.

 PROVOST, F.; FAWCETT, T. *Data Science para negócios*. Rio de Janeiro: Alta Books, 2016.

Referências II

-  SHEPPARD, K. *Introduction to Python for Econometrics, Statistics and Data Analysis*. 2. ed. University of Oxford, 2013. 393 p. Disponível em: <http://onlinelibrary.wiley.com/doi/10.1111/j.1435-5957.2010.00279.x/full>{\% }5Cnpapers3://publication/uuid/790C53F4-2413-4492-8E88-8D54F5A30>.
-  VANDERPLAS, J. *Python Data Science Handbook: Essential Tools for Working with Data*. [S.l.]: O'Reilly, 2017. ISBN 9781491912058.